Agrupamento de consultas para recomendação de índices

Representar a consulta só por FROM e WHERE (método A), comparado ao SQL completo e a atributos estruturais · TPC-H SF1 · PostgreSQL 18

Relatório de experimento · TCC · 6 de outubro de 2026

Resumo. Testamos a hipótese do Expeditus (Duarte et al., 2026): um índice derivado de uma consulta beneficia as demais consultas do mesmo cluster. Reproduzimos o pipeline (Qwen3-Embedding-0.6B, UMAP e HDBSCAN) sobre o TPC-H e comparamos três representações da consulta, mantendo todo o resto igual: o SQL completo (baseline), apenas FROM, JOIN/ON e WHERE (método A) e uma assinatura de atributos com papel e operador (método B). Cada índice proposto foi medido, com tempo real, contra todas as consultas da carga (matriz índice × consulta), e os três agrupamentos foram avaliados sobre essa mesma medição.

Principais resultados

- **O método A foi o melhor dos três.** Foi o mais estável entre sementes (concordância 0,83, contra 0,77 do baseline e 0,50 do B), dividiu em 2 grupos (em vez de 3) as consultas que se beneficiam do índice dominante e reduziu de 9 para 4 as consultas beneficiadas que ficaram fora do cluster do índice.
- **Nenhum método reuniu essas consultas.** O índice em `lineitem(l_partkey)` beneficia Q8, Q9, Q17, Q19 e Q20; a cobertura média ficou em torno de 0,5 nos três métodos, e a Q17 não caiu no mesmo grupo da Q20 em nenhuma das 21 sementes.
- **Avaliar o índice só dentro do cluster esconde regressões.** `orders(o_custkey)` acelera a Q22 (+59%), mas piora a Q8 (−90%), a Q5 (−74%) e a Q7 (−32%).

1. Pergunta

O Expeditus agrupa as consultas por similaridade do texto SQL e cria, para cada cluster, um índice derivado de uma consulta representativa, esperando que ele beneficie as demais do grupo. Se a hipótese vale, o agrupamento reduz o trabalho de ajuste: um índice por grupo, em vez de um por consulta. Este experimento pergunta se a **representação** usada no agrupamento importa e, em especial, se retirar a lista do SELECT, que não tem relação com índices, aproxima os grupos das consultas que de fato compartilham um índice.

2. Montagem experimental

2.1 Base e carga

- TPC-H com fator de escala 1 (`lineitem` com 6.001.215 linhas) em PostgreSQL 18.0, em Docker sobre WSL2 (12 vCPU, 7,6 GB de RAM).
- Esquema TPC-H puro: as 8 chaves primárias e as 8 chaves estrangeiras, sem nenhum índice secundário, view ou consulta adicional.
- Carga gerada pelo BenchBase em power run (Q1–Q22 em série), três rodadas, cada consulta executada duas vezes por rodada (6 execuções). Os planos foram capturados pelo auto_explain e formam um corpus de 22 consultas.
- Sem índice em `lineitem.l_partkey`, a Q17 e a Q20 levam de 3 a 5 horas por execução (a subconsulta correlacionada refaz milhares de varreduras completas de `lineitem`) e somam 99,9% do tempo da carga.

2.2 Representações comparadas

Método	O que vira vetor	Vetorização
Baseline	SQL completo, em minúsculas e com espaços colapsados	Qwen3-Embedding-0.6B (1024 dim.)
A	Só FROM, JOIN/ON e WHERE, incluindo subconsultas e tabelas derivadas, extraídos com o parser do próprio PostgreSQL (pglast)	Qwen3-Embedding-0.6B (1024 dim.)
B	Assinatura de atributos com papel e operador, por exemplo `JOIN:lineitem.l_partkey` e `FILTER:lineitem.l_quantity:<`	TF-IDF binário (67 atributos)

Tabela 1 — As três representações. A extração de A e B roda sobre o SQL original; a bancada de medição sempre reexecuta o SQL original, nunca a representação.

2.3 Agrupamento e escolha do índice

Os três métodos usam exatamente o mesmo agrupamento: UMAP (k = 5, min_dist = 0, 5 dimensões, distância cosseno) seguido de HDBSCAN (tamanho mínimo de cluster 3, min_samples = 2, seleção eom), com semente 42. Para cada cluster, uma heurística extrai índices candidatos dos planos de execução e um modelo de linguagem (Claude) escolhe um índice e a consulta de origem.

2.4 Avaliação: a matriz índice × consulta

A avaliação usada pelo Expeditus mede o índice só nas consultas do cluster e, por isso, não vê quem ele beneficiaria fora dele. Por isso medimos os 6 índices distintos propostos pelos três métodos contra as 21 consultas replicáveis (a Q15 depende de uma view criada pelo próprio BenchBase), numa única sessão de 14,2 minutos:

- linha de base única, sem índice; 5 repetições por medida, a primeira descartada (cache frio) e a mediana das demais;
- com o índice criado, um EXPLAIN indica se o plano de cada consulta o usa; só essas são medidas, e as demais recebem ganho 0, pois o índice não pode tê-las afetado;
- consultas que estouram o tempo limite de 300 s sem o índice (Q17 e Q20) entram com o limite como piso: o ganho real é maior que o reportado.

Uma consulta é **beneficiada** por um índice se o usa e fica pelo menos 10% mais rápida. Para cada cluster e o índice que ele propôs: **cobertura** é a fração das consultas beneficiadas (na carga inteira) que está no cluster; **precisão** é a fração do cluster que é beneficiada; **média geral** é o ganho médio das consultas do cluster, como na Tabela 3 do Expeditus. Como os três métodos são avaliados sobre as mesmas medidas, a diferença entre eles vem do agrupamento, não de variação entre corridas da bancada.

3. Resultados

3.1 Estabilidade entre sementes

Método	Nº de clusters mais frequente	Concordância entre sementes (ARI)	Pares da família l_partkey juntos	Q17 e Q20 juntas
Baseline	4 (14 de 21)	0,77	28%	0%
A (FROM + WHERE)	3 (15 de 21)	0,83	27%	0%
B (atributos)	4 (11 de 21)	0,50	32%	0%

Tabela 2 — 21 sementes (0 a 19 e 42), tamanho mínimo 3. ARI = índice de Rand ajustado entre as partições de pares de sementes (1 = sempre a mesma partição). A família considera as consultas cujo SQL usa l_partkey: Q8, Q9, Q14, Q17, Q19 e Q20.

O método A é o mais estável: com mínimo 3, ele produz 3 clusters em 15 das 21 sementes. O método B, com vetores TF-IDF esparsos e só 22 pontos, é o mais instável. Em nenhum método a Q17 e a Q20 caem juntas com esse tamanho mínimo.

3.2 Os grupos do método A

Grupo	Consultas	Índice escolhido (origem)	Efeito medido
0	Q1, Q3, Q4, Q5, Q6, Q10, Q12, Q13, Q14, Q18, Q21, Q22	I6: `orders (o_orderdate) INCLUDE (o_orderkey, o_custkey, o_shippriority, o_orderpriority)` (Q3)	não beneficia nenhuma consulta; média do grupo 0,0%
1	Q16, Q17, Q19	I2: `lineitem (l_partkey) INCLUDE (l_quantity, l_extendedprice)` (Q17)	beneficia Q17, Q19; média do grupo 64,2%
2	Q2, Q7, Q8, Q9, Q11, Q15, Q20	I3: `lineitem (l_partkey, l_suppkey, l_shipdate) INCLUDE (l_quantity)` (Q20)	beneficia Q8, Q20; média do grupo 23,4%

Tabela 3 — Grupos do método A (rodada 11) e o índice que cada um recebeu. I1 a I6 são os índices da Tabela 4.

Sem o SELECT, o agrupamento separou as consultas que chegam a `lineitem` por pedido e data (grupo 0) das que chegam por peça e fornecedor (grupos 1 e 2). Toda a família `l_partkey` ficou nos grupos 1 e 2; no baseline, a Q17 estava junto de Q1, Q6 e Q14, que só filtram por data. Em compensação, o grupo 0 virou um conjunto de 12 consultas para o qual o índice escolhido não beneficia ninguém.

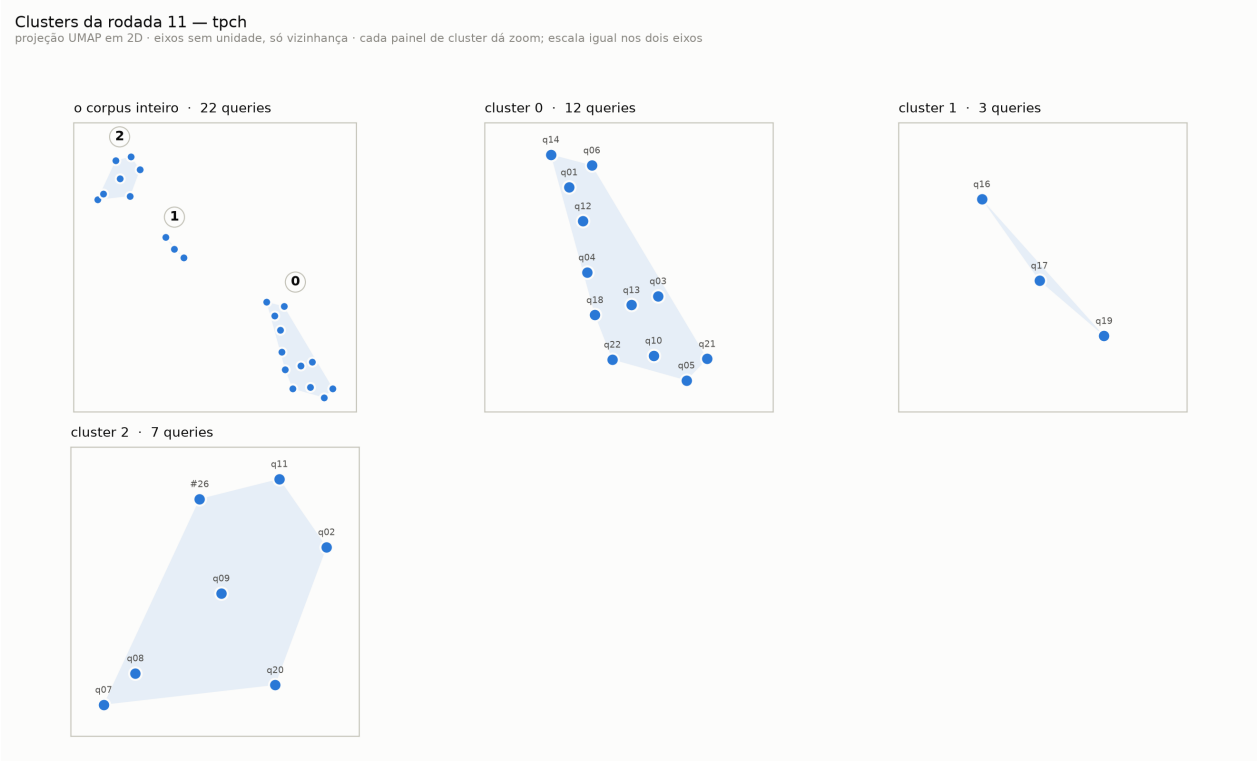


Figura 1 — Projeção UMAP em 2D dos grupos do método A, um painel por grupo. O ponto “#26” é a Q15, cuja versão do BenchBase não casa com o texto do arquivo de referência.

3.3 A matriz índice × consulta

	Índice	Tamanho	Proposto por
I1	`lineitem (l_partkey)`	43,0 MB	Baseline, B
I2	`lineitem (l_partkey) INCLUDE (l_quantity, l_extendedprice)`	223,2 MB	A
I3	`lineitem (l_partkey, l_suppkey, l_shipdate) INCLUDE (l_quantity)`	232,1 MB	Baseline, A, B
I4	`orders (o_custkey)`	12,0 MB	Baseline, B
I5	`orders (o_custkey, o_orderdate) INCLUDE (o_orderkey)`	45,1 MB	B
I6	`orders (o_orderdate) INCLUDE (o_orderkey, o_custkey, o_shippriority, o_orderpriority)`	71,0 MB	A

Tabela 4 — Os 6 índices distintos propostos pelos três métodos, com o tamanho medido em SF1.

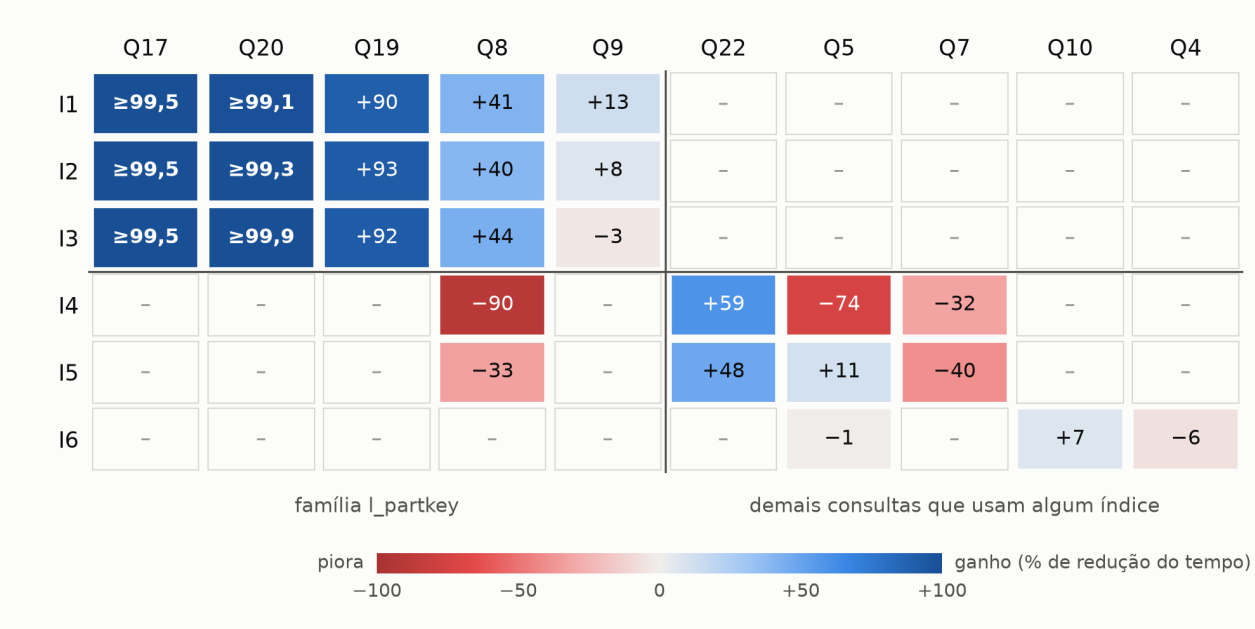


Figura 2 — Ganho medido de cada índice (linhas) em cada consulta que usa algum deles (colunas). "–" indica que o plano não usa o índice (ganho considerado 0). Nas células "≥", a consulta estourou 300 s sem o índice e o valor é um piso. As 11 consultas que não usam nenhum dos índices foram omitidas.

Duas estruturas aparecem. Primeiro, os três índices em `lineitem` que começam por `l_partkey` (I1 a I3) beneficiam o mesmo conjunto de consultas: Q17, Q20, Q19, Q8 e, em menor grau, Q9. O índice composto I3 atende a todas e leva a Q20 de mais de 5 minutos para 0,1 s. Segundo, os índices em `orders(o_custkey…)` (I4 e I5) aceleram a Q22, mas o otimizador passa a escolher planos piores para Q5, Q7 e Q8.

3.4 Comparação entre os métodos

Métrica	Baseline	A (FROM + WHERE)	B (atributos)
Clusters	4	3	4
Cobertura média	0,51	0,50	0,54
Precisão média	0,35	0,33	0,31
Média geral do ganho	18,6%	29,2%	18,1%
Beneficiadas fora do cluster (soma)	9	4	7
Grupos em que a família l_partkey ficou	3	2	2 + 1 em ruído
Índices em l_partkey propostos	3	2	2

Tabela 5 — Os três métodos avaliados sobre a mesma matriz (limiar de 10%). A família l_partkey medida é {Q8, Q9, Q17, Q19, Q20}.

O método A vence em estabilidade, em fragmentação da família, em índices redundantes e em consultas beneficiadas perdidas fora do cluster. Em cobertura e precisão médias, os três empatam: diferenças de 0,04 equivalem a menos de uma consulta mudando de grupo. A média geral mais alta do A não indica, sozinha, um agrupamento melhor: ela vem em boa parte de ele ter só 3 grupos, um deles pequeno e contendo a Q17.

3.5 Onde ficou a família l_partkey

Consulta	Baseline	A (FROM + WHERE)	B (atributos)
Q17	grupo 0	grupo 1	grupo 0
Q19	grupo 0	grupo 1	grupo 0
Q8	grupo 2	grupo 2	ruído
Q9	grupo 2	grupo 2	grupo 2
Q20	grupo 3	grupo 2	grupo 2
Grupos distintos	3	2	2 + ruído

Tabela 6 — Grupo de cada consulta beneficiada pelo índice dominante. A composição de cada grupo está no Apêndice A.

A A junta a Q20 com a Q8 e a Q9, mas mantém a Q17 e a Q19 à parte. Cada pedaço recebe o seu próprio índice em `l_partkey`, quando um só (I3) atenderia às cinco consultas. Em SF1 cada índice desses ocupa de 43 a 232 MB.

3.6 Escolher o índice pelo cluster inteiro

Método	Índice escolhido pelo modelo	Melhor dos 6 para o cluster inteiro	Diferença
Baseline	18,6%	21,0%	+2,4 p.p.
A (FROM + WHERE)	29,2%	31,6%	+2,4 p.p.
B (atributos)	18,1%	21,5%	+3,4 p.p.

Tabela 7 — Média geral do ganho com o índice escolhido e com o melhor dos 6 índices medidos para cada cluster (ou nenhum índice, quando nenhum ajuda).

Escolher o índice pelo efeito no cluster inteiro, como descreve a Seção 3.5 do Expeditus, melhora a média em 2 a 3 pontos percentuais. O ganho vem sobretudo de evitar escolhas que prejudicavam o próprio grupo: no baseline, trocar `orders(o_custkey)` por `orders(o_custkey, o_orderdate)` deixa de piorar a Q5; no B, para o grupo {Q5, Q7, Q21}, a melhor opção é não criar índice. Isso não corrige a fragmentação: cada pedaço da família continua recebendo um índice próprio.

3.7 Recorte sem Q2, Q17 e Q20

	Baseline	A (FROM + WHERE)	B (atributos)
Cobertura média	0,72	0,83	0,61
Precisão média	0,23	0,22	0,23
Média geral do ganho	11,2%	7,8%	9,3%

Tabela 8 — Os três métodos reagrupados sem Q2, Q17 e Q20, com o melhor dos 6 índices por cluster.

Kossmann et al. (2020) excluem essas três consultas porque o custo delas domina a carga. Repetindo o recorte, a cobertura sobe, mas por um efeito artificial: sem a Q17 e a Q20, cada índice beneficia menos consultas, e conjuntos pequenos são fáceis de cobrir. A precisão e a média caem, e a família restante {Q8, Q9, Q19} continua dividida em todos os métodos.

4. Discussão

Por que o método A ajuda. No baseline, o que aproximava consultas como Q1, Q6, Q14 e Q17 era o SELECT (`sum(l_extendedprice * (1 - l_discount))`), que nenhum índice toca. Sem ele, os grupos passam a refletir por onde cada consulta chega às tabelas, e o resultado fica mais estável e menos fragmentado.

Por que a família continua partida. As cinco consultas beneficiadas pelo índice dominante compartilham um único caminho de acesso (`lineitem` por `l_partkey`) e diferem em todo o resto: Q17 e Q19 são orientadas a peças, Q20 a fornecedores e estoque, Q8 e Q9 são junções de seis tabelas. Qualquer similaridade global, textual ou estrutural, separa essas consultas. Além disso, um agrupamento por partição põe cada consulta em um só grupo, enquanto os conjuntos de consultas beneficiadas se sobrepõem: a Q8 ganha 40% com I1 a I3 e perde 90% com I4.

A avaliação intra-cluster esconde prejuízos. Medido só dentro do seu cluster, `orders(o_custkey)` parecia quase neutro; a matriz mostra que ele piora em 90% uma consulta de outro cluster. Como foi o próprio otimizador que escolheu esses planos, por estimá-los mais baratos, uma avaliação por custo estimado (como a de Kossmann et al.) tenderia a contá-lo como melhoria.

Relação com a literatura. O Expeditus também relata piora na Q5 (+49%) e na Q8 (+83%). Kossmann et al. observam o mesmo domínio de Q2, Q17 e Q20, e as oito consultas que eles apontam como não afetadas por índices (1, 3, 6, 7, 10, 13, 14 e 16) também não se beneficiaram aqui. As diferenças restantes (Q2, Q4, Q18 e Q21) se explicam pela linha de base deles não ter chaves primárias, que aqui já atendem a esses acessos.

5. Limitações

- 22 consultas de um único benchmark, em SF1. O agrupamento é sensível à semente; isso foi mitigado medindo a estabilidade em 21 sementes, mas a análise principal usa a semente 42.
- A escolha do índice por um modelo de linguagem não é determinística, e a matriz cobre só os 6 índices propostos.
- O limiar de 10% para “beneficiada” é uma escolha nossa. A Q15 ficou fora da medição. Para a Q17 e a Q20, os ganhos são pisos.
- Os tempos do corpus da 3ª rodada foram afetados por uma hibernação do computador. Eles influenciam apenas a ordenação dos candidatos; a matriz foi medida numa sessão separada.
- A base mantém as chaves primárias, por realismo, o que a diferencia da montagem de Kossmann et al.

6. Próximos passos

- Controle com agrupamento aleatório (mesmos tamanhos de cluster), para saber se algum método supera o acaso. É calculável sobre a matriz atual, sem novas medições.
- Inverter a ordem: selecionar os índices para a carga inteira e definir os grupos como “as consultas atendidas pelo mesmo índice”, como limite superior para qualquer agrupamento.
- Comparar com algoritmos clássicos (Extend e DB2Advis, da plataforma de Kossmann et al.), medindo os índices que eles recomendam na mesma matriz.
- Agrupamento com sobreposição, permitindo que uma consulta pertença a mais de um grupo.

Referências

DUARTE, M. G.; MARINHO, J. C. L.; BRAYNER, A. R. A.; FRANCO, W. Expeditus: Database Optimization Based on Workload Analysis with Structural Query Clustering. In: Simpósio Brasileiro de Banco de Dados (SBBD), 41., 2026, São Carlos. Anais, p. 510–522.

KOSSMANN, J.; HALFPAP, S.; JANKRIFT, M.; SCHLOSSER, R. Magic mirror in my hand, which is the best in the land? An Experimental Evaluation of Index Selection Algorithms. PVLDB, v. 13, n. 11, p. 2382–2395, 2020.

McINNES, L.; HEALY, J.; MELVILLE, J. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. arXiv:1802.03426, 2018.

McINNES, L.; HEALY, J.; ASTELS, S. hdbscan: Hierarchical density based clustering. Journal of Open Source Software, v. 2, n. 11, p. 205, 2017.

Qwen3-Embedding-0.6B. Disponível em: https://huggingface.co/Qwen/Qwen3-Embedding-0.6B.

BenchBase. Disponível em: https://github.com/cmu-db/benchbase.

pglast: PostgreSQL Languages AST and statements prettifier. Disponível em: https://github.com/lelit/pglast.

TRANSACTION PROCESSING PERFORMANCE COUNCIL. TPC Benchmark H (TPC-H).

Apêndice A — Composição dos grupos

Baseline (rodada 2)

Grupo	Consultas	Índice (origem)	Beneficia	Piora
0	Q1, Q6, Q14, Q17, Q19	I1 (Q17)	Q17, Q19	—
1	Q3, Q4, Q5, Q10, Q12, Q13, Q18, Q22	I4 (Q3)	Q22	Q5
2	Q7, Q8, Q9	I1 (Q8)	Q8, Q9	—
3	Q2, Q11, Q15, Q16, Q20, Q21	I3 (Q20)	Q20	—

A (FROM + WHERE) (rodada 11)

Grupo	Consultas	Índice (origem)	Beneficia	Piora
0	Q1, Q3, Q4, Q5, Q6, Q10, Q12, Q13, Q14, Q18, Q21, Q22	I6 (Q3)	—	—
1	Q16, Q17, Q19	I2 (Q17)	Q17, Q19	—
2	Q2, Q7, Q8, Q9, Q11, Q15, Q20	I3 (Q20)	Q8, Q20	—

B (atributos) (rodada 12)

Grupo	Consultas	Índice (origem)	Beneficia	Piora
ruído	Q2, Q4, Q8, Q12	—	—	—
0	Q6, Q14, Q17, Q19	I1 (Q17)	Q17, Q19	—
1	Q1, Q3, Q10, Q13, Q18, Q22	I4 (Q22)	Q22	—
2	Q9, Q11, Q15, Q16, Q20	I3 (Q20)	Q20	—
3	Q5, Q7, Q21	I5 (Q7)	Q5	Q7